Cipherops

PQC DNSSEC MTL Benchmark Using a Reference Implementation (PS-036)

How it works: architecture, pipeline, measurements and results

Akash Joshi · A3-PS036-TC161
akashjoshi2023@gmail.com

AIORI Hackathon 3

October 8, 2026

Say:

Hello everyone. This session walks through Cipherops, our benchmark for problem statement PS-036: post-quantum DNSSEC with Merkle Tree Ladder mode, measured with the supplied reference implementation. I will explain why the problem matters, how the system is built, what happens in each of the seven stages of a run with real input and output values, what every measurement means, and what the results show. All numbers come from our quick demo run of October 7: three zone sizes, five signing modes, three trials each.

Slide 2 / 23

Agenda

From the problem to the numbers, in the order a run executes.

1. **The problem**: post-quantum signatures in DNS
2. **MTL mode** in one picture
3. **What PS-036 asks** and how we measure it
4. **Architecture**: components, container, data flow
5. **How to run it**
6. **The 7 stages of a run**, with real inputs and outputs
7. **Results**: signing, verification, answer sizes, bytes on the wire, cache
8. **The result in one table**
9. **What is measured vs modelled**, limitations
10. **Reproduce and learn**: commands, trace script, docs

Say:

Here is the plan. First the problem and the one idea MTL brings. Then what PS-036 asks us to measure and how Cipherops is built to measure it. Then the core of the talk: the seven stages of a run, each with its inputs, what happens, and what comes out. Then the results, one slide per measurement, a single summary table, an honest look at what is measured versus modelled, and finally how anyone can reproduce and inspect every number.

Slide 3 / 23

Post-quantum signatures do not fit today's DNS

Every DNSSEC answer carries a signature. Post-quantum signatures are huge and slow to make.

	Ed25519 / ECDSA	RSA-2048	SLH-DSA-SHA2-128s (post-quantum)
Signature in every answer	64 B	256 B	**7,856 B**
Sign one record set	≈ 30 µs	≈ 0.6 ms	**≈ 0.5 s**
Small answer (one IP address)	178 B	370 B	**7,970 B**
Fits a 1,232-byte UDP answer?	yes	yes	**never → truncated, retried over TCP**

- Re-signing a 10,005-record-set zone with SLH-DSA takes about **80 minutes** on one core.
- **MTL mode** (Merkle Tree Ladder) promises to fix size and speed: sign the whole zone **once**, give each answer a small hash path. **PS-036**: measure whether it really does, with the reference implementation.

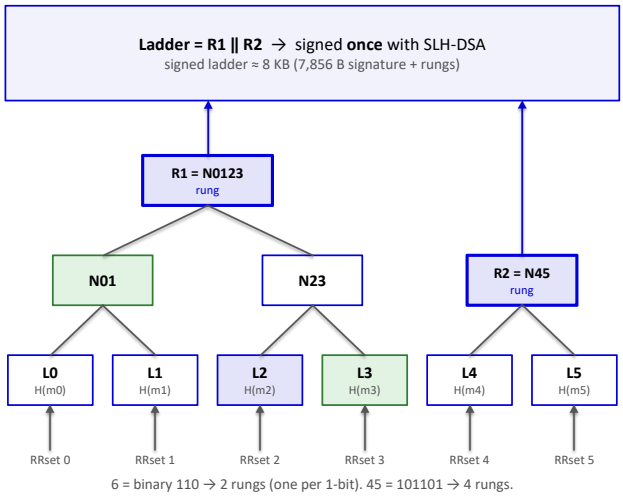
Say:

DNSSEC attaches a signature to every answer. Classical signatures are tiny, 64 bytes for Ed25519 or ECDSA. The post-quantum scheme we test, SLH-DSA-SHA2-128s from FIPS 205, has 7,856-byte signatures and takes about half a second to sign one record set. Two consequences. First, an answer for a single IP address grows from 178 bytes to almost 8 kilobytes, which never fits the 1,232-byte UDP buffer recommended since DNS Flag Day 2020; every lookup is truncated and has to be repeated over TCP. Second, signing a ten-thousand-record zone takes about 80 minutes. MTL mode claims to solve both by signing once per zone. PS-036 asks us to measure that claim with Verisign's reference implementation, and to defer if that implementation does not work.

Slide 4 / 23

MTL mode in one picture

Example zone with 6 record sets. Real zones: 45, 125, 405 (quick demo) up to 10,005.



6 = binary 110 → 2 rungs (one per 1-bit). 45 = 101101 → 4 rungs.

Key terms

- **Condensed signature** of RRset 2 = the hash path to its rung: **L3, N01** (green). That, plus a 1-byte type, is what goes in its RRSIG: **156–204 B** in our zones instead of 7,856 B.
- **Signed ladder** = the rungs + one SLH-DSA signature. One per zone, about **8 KB**. A resolver fetches it once (EDNS option mtl-mode-full, over TCP) and caches it.
- **Full signature** = condensed + signed ladder. Used only when the resolver has no ladder yet.
- Verify with a cached ladder: recompute L2 → N23 → R1, compare with the ladder: **hashes only**.

Say:

This is the whole idea of MTL in one picture. Each record set's signing input is hashed into a leaf of a Merkle tree. Leaves are paired and hashed upwards. With 6 record sets, 6 is binary 1-1-0, so the tree ends in two complete subtrees whose tops are called rungs: R1 covers leaves 0 to 3, R2 covers 4 and 5. The rungs together are the ladder, and only the ladder is signed with SLH-DSA, once for the whole zone. The answer for record set 2 carries only the condensed signature: the sibling hashes on its path to the rung, here L3 and N01, shown in green. A resolver that already holds the verified ladder recomputes L2, then N23, then R1 and checks that R1 is in the ladder: a few hash operations instead of an SLH-DSA verification. The first time, it needs the ladder itself, which it fetches once with the mtl-mode-full option. In our zones the condensed signature is 156 to 204 bytes and the signed ladder about 8 kilobytes.

Slide 5 / 23

What PS-036 asks, and how we measure it

Every requirement maps to one stage of the run and one output file.

PS-036 requirement	How Cipherops does it	Where the number is
Use the supplied reference implementation	Verisign MTL library, unmodified, pinned commit 7cef019	environment.json
Defer if it or its fixtures fail	Go/no-go gate: Verisign test suite, tool round trip, harness smoke test; NO-GO stops the run (exit 3)	preflight.json
One PQC algorithm, with and without MTL	SLH-DSA-SHA2-128s in MTL mode vs signing every RRset; Ed25519, ECDSA, RSA as references	trials.csv
Signing time, verification time	C harness times every library call in nanoseconds; warm and cold verification	trials.csv, ops.parquet
Transmitted bytes	Real DNS answers with the real signatures; UDP, truncation, TCP; 301-lookup stream	responses.csv, workload.csv
Cache usage	Resolver cache measured (dnspython LRUCache); draft §12 formulas alongside	cache.csv, sizes.csv
Several dataset sizes, repeated trials	3 zone sizes; 3 trials (5 in the full run); medians with 95% bootstrap CI	summary.json, report

Say:

PS-036 is specific, so we built the benchmark around its list. Use the reference implementation: we link Verisign's MTL library unmodified at a pinned commit. Defer if it fails: a go/no-go gate runs Verisign's own tests before anything is measured. One post-quantum algorithm with and without MTL: SLH-DSA in MTL mode against SLH-DSA signing every record set, with three classical algorithms as reference points. Then the four measurements: signing time and verification time come from a C harness that times each library call; transmitted bytes come from real DNS messages built around the real signatures; cache usage is measured in a real resolver cache. Three dataset sizes and repeated trials, summarised with medians and confidence intervals. The right column tells you which output file holds each number.

Slide 6 / 23

Solution Architecture & Design

UML component view: one pinned container runs every stage and calls the unmodified reference libraries.

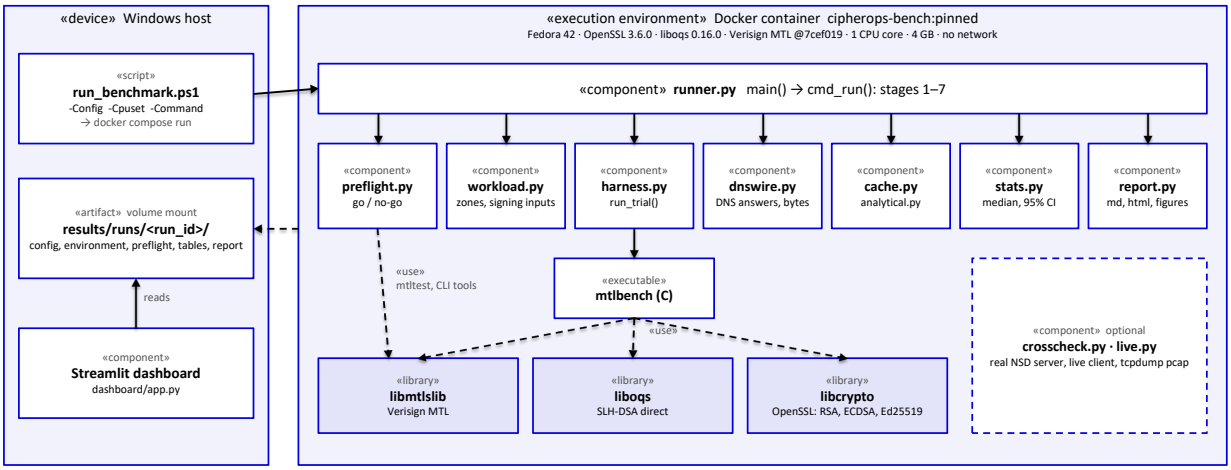


Figure: System Architecture UML Diagram

Say:

This is the component view. On the Windows host there are only three things: the PowerShell script that starts a run, the results folder, and the Streamlit dashboard that reads it. Everything else runs in one pinned Docker container: Fedora 42 with OpenSSL 3.6.0, liboqs 0.16.0 and Verisign's MTL library, limited to one CPU core, 4 GB and no network, so timings are comparable. Inside, runner.py drives seven Python components, one per stage. The only component that touches cryptography is harness.py, which launches mtlbench, our small C program. mtlbench calls the three libraries directly: Verisign MTL for MTL mode, liboqs for SLH-DSA without MTL, OpenSSL for the classical algorithms. We implement no cryptography ourselves. The preflight component also uses Verisign's own test binary and command-line tools. The dashed box is optional: a cross-check against a real NSD DNS server and a live packet capture. Results reach Windows through a volume mount.

Slide 7 / 23

Data Flow Diagram

Level-1 DFD of one run: from the configuration to the report.

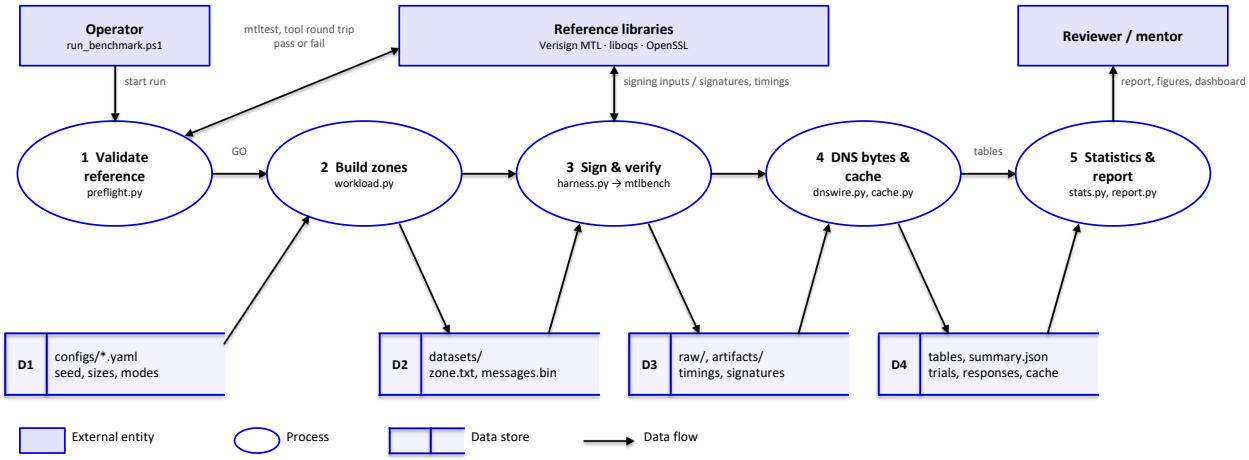


Figure: Data Flow Diagram (DFD)

Say:

The same system as data flow. The operator starts a run. Process 1 validates the reference library against its own tests; only on GO do we continue. Process 2 builds the zones from the configuration, data store D1, and writes the signing inputs, D2. Process 3 sends those inputs to the reference libraries and gets back real signatures and timings, D3. Process 4 builds DNS answers around those signatures and computes bytes and cache use. Process 5 turns all tables, D4, into statistics, the report and the dashboard for reviewers. Each data store is a folder or file inside the run directory, so every number can be traced back.

Slide 8 / 23

Pinned environment and how to run it

One image, one command. The host needs only Docker and git.

Component	Version (pinned)
Base image	Fedora 42 (fedora-minimal)
OpenSSL	3.6.0
liboqs	0.16.0 (Release)
Verisign MTL	commit 7cef019, -O2
Harness	mtlbench (C), built in the image
Python	3.13, dnspython 2.8, pandas, numpy
Limits	1 CPU core (cpuset 2), 4 GB, no network

Commands (Windows PowerShell, repository root)

Full PS-036 run (50 / 500 / 5,000 names, 5 trials)

```
.\scripts\run_benchmark.ps1
```

Quick demo (20 / 60 / 200 names, 3 trials)

```
.\scripts\run_benchmark.ps1 -Config configs/quick.yaml
```

Go/no-go gate only

```
.\scripts\run_benchmark.ps1 -Command preflight
```

Dashboard

```
streamlit run dashboard/app.py
```

The script builds the image, records git SHA and image ID, then runs python3 -m cipherops.runner run inside the container. Results appear in results\runs\<run_id>\.

Say:

Reproducibility is the first design rule. Every library is pinned to an exact tag or commit and built into one Docker image; the Verisign library is compiled with -O2 like liboqs. Each trial runs on a single dedicated CPU core with 4 GB of memory and no network, so nothing else competes with the measurement. One PowerShell script builds the image and starts the run; the same script runs only the go/no-go gate. A run writes a timestamped folder under results/runs, and the dashboard reads those folders. Our quick demo used configs/quick.yaml; the full PS-036 run uses the default scenarios file.

Slide 9 / 23

One run = 7 stages

runner.py cmd_run() executes them in order. Values from the quick demo run.

Stage	Code	Input	Output
1 Config + environment	load_config, envinfo	quick.yaml: seed 20261005, 3 trials, 5 modes, 20 / 60 / 200 names	config.yaml, environment.json
2 Go/no-go gate	preflight.py	Verisign library, tests, tools; mtlbench	preflight.json: GO (9 checks)
3 Zones + signing inputs	workload.py	names, seed, modes	45 / 125 / 405 RRsets; 15 × messages.bin
4 Sign + verify	harness.py → mtlbench	15 messages.bin files	45 trials: trials.csv, raw/, artifacts/ (signatures)
5 DNS bytes + cache	dnswire.py, cache.py	zones, real signatures, 301 lookups	responses, workload, cache, sizes .csv
6 Statistics	stats.py, summarise	45 trial rows + stage 5 tables	summary.json (median, 95% CI)
7 Report	report.py	the run folder	report.md / .html, 5 figures

Say:

A run is seven stages executed in order by cmd_run in runner.py. Stage 1 loads the configuration and records the environment. Stage 2 is the go/no-go gate. Stage 3 builds the DNS zones and, for every algorithm, the exact bytes each signature must cover. Stage 4 is the cryptography: our C harness signs and verifies everything, 45 times in the quick demo. Stage 5 turns signatures into DNS traffic and counts bytes and cache use. Stage 6 computes medians and confidence intervals. Stage 7 writes the report. The next slides take each stage in turn with its real values.

Slide 10 / 23

Stages 1–2: configuration, environment, go/no-go

Before any number is produced we record what we run on, and prove the reference library works.

Stage 1 – inputs and record

- **configs/quick.yaml**: seed 20261005, 3 trials + 1 warm-up, 5 modes, zones of 20 / 60 / 200 names, 16 cold MTL verifications per trial, direct SLH-DSA capped at 60 / 100 signatures, 300 lookups.
- **Run folder**: results/runs/**20261007T045213Z-quick-demo/**
- **environment.json**: Intel Core Ultra 7 155U, pinned core [2], WSL2 kernel, OpenSSL 3.6.0, liboqs 0.16.0, MTL 7cef019 (-O2), git SHA, image ID, SHA-256 of mtlbench.

Stage 2 check	Result
Library files installed	ok
Verisign test suite (mtltest)	89 passed, 0 failed
mtlkeygen → mtlsign → mtlverify, 16 msgs + 1 tampered	all verified, tamper rejected
mtlbench smoke test, 6-name zone, 5 modes	all pass, tamper rejected
Bundled vector scheme (warning only)	ML-DSA-44-MTL
Verdict	**GO**

NO-GO on any critical check → the run prints the verdict and stops with exit code 3. No benchmark numbers are produced (PS-036: defer).

Say:

Stage 1 reads the configuration, creates a timestamped run folder and writes two files: the configuration as used, and environment.json, which records the CPU, the pinned core, every library version, the Git commit, the Docker image ID and a hash of our harness. Anyone reading a number later can see exactly what produced it. Stage 2 is the go/no-go gate PS-036 asks for. It checks the library is installed, runs Verisign's own test suite, 89 tests, runs Verisign's command-line tools end to end on 16 messages and checks a tampered message is rejected, and runs our harness on a tiny zone in all five modes. All passed, verdict GO. Had any critical check failed, the run would stop with exit code 3 and produce no numbers.

Slide 11 / 23

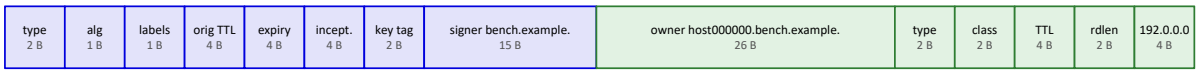
Stage 3: zones and signing inputs

Deterministic DNS zones, then the exact bytes each signature must cover (RFC 4034 §3.1.8.1).

Zone	Names	RRsets	apex	small	medium	large	NSEC3
small	20	45	4	14	3	3	21
medium	60	125	4	32	17	11	61
large	200	405	4	117	70	13	201

- Zone **bench.example.**, seed + dataset index
- **small** 60%: one A / AAAA · **medium** 30%: 3 MX or 4 AAAA · **large** 10%: TXT 1–2 KB
- RRsets = 2 × names + 5 (4 apex, 1 per name, NSEC3 chain)

One signing input (here host000000 A, 73 bytes)



RRSIG fields, signature left empty: 33 B

the RRset in canonical form: 40 B

- Per mode only the algorithm byte, key tag and DNSKEY change, so each of the **15** (zone, mode) pairs gets its own **messages.bin**: small/MTL = 45 messages, 9,311 B. Keys do not exist yet: DNSKEY holds a placeholder of the right length.
- Direct SLH-DSA is capped at 60 / 100 evenly spaced RRsets (**messages_sample.bin**); its zone total is extrapolated.

Say:

Stage 3 builds three DNS zones from the seed, so they are identical on every run. Each name gets one record set: 60 percent small, a single IP address; 30 percent medium, three mail servers or four IPv6 addresses; 10 percent large, a 1 to 2 kilobyte text record. Add the four apex record sets and the NSEC3 chain for denial of existence: 20 names become 45 record sets, 200 names become 405. Then, for every algorithm, we compute what DNSSEC actually signs: the RRSIG fields with an empty signature, 33 bytes here, followed by the record set in canonical form. For host000000's A record that is 73 bytes. Because the algorithm number and key tag are inside those bytes, every mode gets its own file, 15 in total. No keys exist yet; they are created inside each trial. Direct SLH-DSA is too slow to sign every record of the bigger zones, so it signs an evenly spaced sample and we extrapolate.

Slide 12 / 23

Stage 4: sign and verify (one trial)

45 trials = 3 trials × 3 zones × 5 modes, shuffled order, one new process and new key pair each.



MTL trial, small zone (45 RRsets)	Calls	Time
Key generation	1	68.6 ms
Add each RRset as a leaf (hash)	45	3.9 µs each
Sign the ladder (SLH-DSA)	**1**	**486 ms**
Extract condensed signatures	45	0.24 µs each
Verify the ladder	1	441 µs
Verify an answer, ladder cached	45	**2.4 µs each**
Verify a full signature (cold)	16	604 µs each
Tamper: message, signature, ladder	3	all rejected

Other modes, same zone

- **SLH-DSA direct**: 45 × sign ≈ **0.50 s**, 45 × verify ≈ 0.47 ms.
- **Ed25519**: 45 × sign ≈ 31 µs, verify ≈ 93 µs. ECDSA signatures converted to DNSSEC's 64-byte form.
- **Outputs**: raw/*.json.gz (45, exact JSON), artifacts/ (trial 0 signatures, used in stage 5), trials.csv (45 rows), ops.parquet (16,899 timed calls).
- All 45 trials: exit 0, 0 failures, every tamper test rejected.

Say:

Stage 4 is where the cryptography happens. For each of the 45 trials, Python starts our C program mtlbench with one messages file. mtlbench makes a fresh key pair, signs every message, verifies every message, and times each library call with a nanosecond clock. For MTL on the small zone: 45 cheap hash operations add the record sets as leaves; the ladder is signed once with SLH-DSA, 486 milliseconds, which is almost all of the signing cost; the condensed signatures are extracted. On the verifier side it builds a separate verifier from the public key only, like a resolver, verifies the ladder once, then verifies each answer in about 2.4 microseconds. It also does 16 cold verifications with the full signature, and flips one bit in a message, a signature and the ladder: all three must be rejected. Direct SLH-DSA instead signs every record at half a second each. The output JSON is stored raw, summarised into one row of trials.csv, and the first trial's signatures are kept for stage 5.

Slide 13 / 23

Stage 5: DNS answers, bytes on the wire, cache

Real DNS messages (dnspython) with the real signatures inside. No verification here: sizes and traffic.

host000000 A, buffer 1,232 B	Answer	Signature	Transport	Bytes both ways
Ed25519	178 B	64 B	UDP	231
SLH-DSA + MTL (ladder cached)	271 B	157 B	UDP	324
SLH-DSA + MTL, first contact (full)	8,295 B	8,181 B	TCP	8,356
SLH-DSA direct	7,970 B	7,856 B	UDP → TC=1 → TCP	8,133

Transport model (dnswire.exchange)
fits: query + answer
too big: query + TC reply + (2 + query) + (2 + answer)
MTL full: (2 + query) + (2 + answer) over TCP

- **responses.csv**: every answer measured once (DNSKEY, each name, 50 NXDOMAIN) at 1,232 and 4,096 B.
- **cache.csv**: answers loaded into dnspython's cache and measured.

The lookup stream (workload.csv)

- **301 lookups**: DNSKEY first, then 300; 10% for names that do not exist.
- Popularity follows **Zipf (s = 1.1)**: rank r gets weight 1/(r+1)^1.1, like real DNS traffic.
- A repeat lookup is served from the **resolver cache**: only **48 / 74 / 106** answers are sent.
- MTL: after the first answer, **one** mtl-mode-full fetch of the ladder over TCP.

Say:

Stage 5 answers a different question from stage 4: not how long the cryptography takes, but what travels over the network and what a resolver stores. We build real DNS queries and answers with dnspython and put the real signatures from stage 4 inside. For host000000's A record the answer is 178 bytes with Ed25519, 271 with MTL, and 7,970 with direct SLH-DSA, which does not fit 1,232 bytes, so the server sends an empty truncated reply and the client asks again over TCP. The transport model adds those exchanges up. Then we replay a realistic stream of 301 lookups: the DNSKEY first, popular names much more often than rare ones following Zipf's law, 10 percent non-existent names. Repeats are served from cache, so only 48 answers are sent for the small zone. For MTL the resolver fetches the ladder once over TCP after its first answer. Finally the received answers are loaded into dnspython's resolver cache and measured.

Stages 6–7: statistics and report

Medians over trials with a 95% bootstrap confidence interval; one report per run.

- **Input**: 45 trial rows (3 values per zone × mode) and the stage 5 tables.
- **stats.summarize**: n, mean, **median**, std, p05, p95, min, max.
- **stats.bootstrap_ci**: resample the 3 trial values 2,000 times, take each median, keep the 2.5th–97.5th percentile.
- **summary.json**: trials, transmitted bytes, UDP fit per record class. Example: large MTL signing median **0.516 s**, CI [0.472 s, 0.575 s].
- **all_checks_passed** combines exit codes, failures and tamper tests per zone × mode.

Stage 7 – report.write(run)

- **report.md / report.html**: problem, method, all tables, discussion, limitations, appendices with config, environment and gate.
- **figures/**: sign, verify, responses, workload, cache (next slides).
- **Streamlit dashboard** reads the same run folder.
- Exit code: 0 all checks passed · 1 a trial failed · 3 NO-GO.

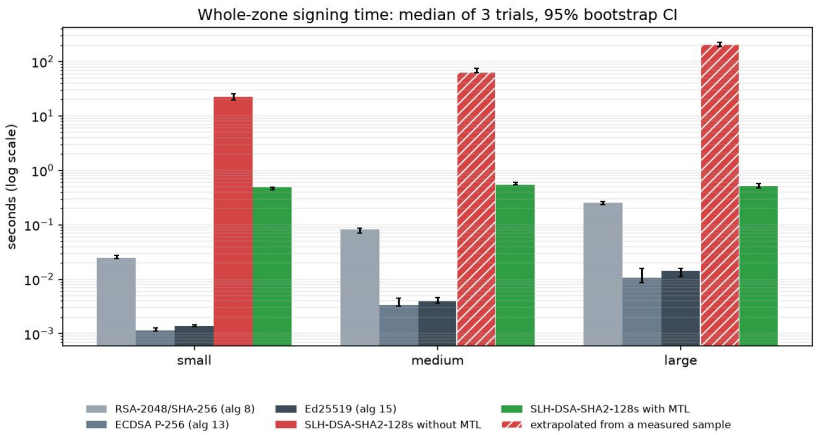
Say:

Stage 6 turns repeated trials into statistics. For every zone and mode we take the trial values and report the median, because a single slow trial should not move the result, and a 95 percent bootstrap confidence interval: resample the trial values two thousand times, take the median of each resample, and keep the middle 95 percent. For example, signing the large zone with MTL has a median of 0.516 seconds with an interval from 0.47 to 0.58. Everything goes into summary.json. Stage 7 writes a complete report in Markdown and HTML with the figures you will see next, and the Streamlit dashboard reads the same folder. The process exit code tells automation whether every check passed.

Slide 15 / 23

Signing time: MTL signs the zone once

Whole-zone signing time per mode and zone size.



Large zone (405 RRsets)

- SLH-DSA direct: **≈ 209 s** (100 signed at 0.52 s, × 405)
- SLH-DSA + MTL: **0.52 s** – one ladder signature + 405 hashes
- Ed25519 0.014 s · ECDSA 0.011 s · RSA 0.25 s
- MTL stays ≈ 0.5 s at every zone size: cost is per zone, not per record.

Quick demo run, log scale. Source: trials.csv, median of 3 trials, whiskers = 95% CI, hatched = extrapolated.

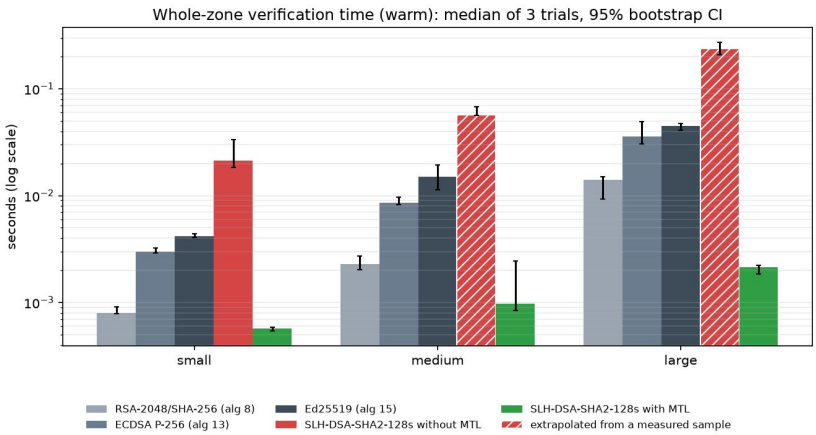
Say:

Signing time for the whole zone, on a log scale. The red bars are SLH-DSA without MTL; hatched means extrapolated from the measured sample. It grows with the zone: 22 seconds for 45 record sets, about 209 seconds for 405, and it would be about 80 minutes for the full 10,005. The green bars are MTL: about half a second at every size, because the expensive SLH-DSA signature is made once per zone and each record only adds a hash. That is about 400 times faster on the large zone, and the gap grows with zone size. Classical algorithms remain faster still, which is expected.

Slide 16 / 23

Verification time: hashes instead of SLH-DSA

Whole-zone verification with a cached ladder (warm).



Quick demo run, log scale. Source: trials.csv, median of 3 trials, whiskers = 95% CI.

Per answer, large zone

- Warm: SLH-DSA direct **588 µs** · MTL **5.3 µs** · Ed25519 111 µs
- Cold (first answer, no ladder): direct 511 µs · MTL **513 µs** · Ed25519 103 µs
- MTL pays one SLH-DSA verification per zone (the ladder), then only hashes.

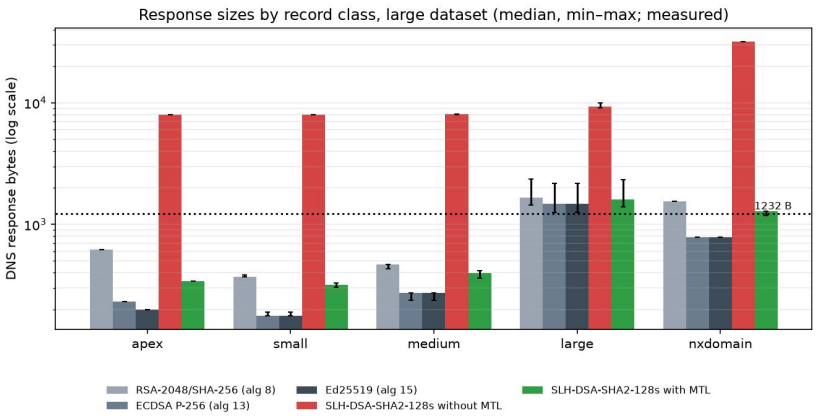
Say:

Verification, again whole zone and log scale. Once a resolver holds the verified ladder, every MTL answer is checked with a few hashes: 5.3 microseconds per answer on the large zone, including the one ladder verification spread over all answers. That is about 100 times faster than direct SLH-DSA and even faster than Ed25519 or ECDSA. The first answer is different: with nothing cached the resolver must verify the ladder's SLH-DSA signature, 513 microseconds, the same as one direct SLH-DSA verification. So MTL moves the post-quantum cost to the first contact with a zone.

Slide 17 / 23

Answer sizes: MTL fits in UDP again

Every DNS answer measured, large zone, by record class.



Share of answers ≤ 1,232 B

- Small answers: direct **0%** (7,970 B) · MTL **100%** (319 B) · Ed25519 100% (178 B)
- All answers, large zone: direct 0% · MTL 78% · Ed25519 95%
- Over the line for MTL: large TXT records and some NXDOMAIN (2–4 signatures each).

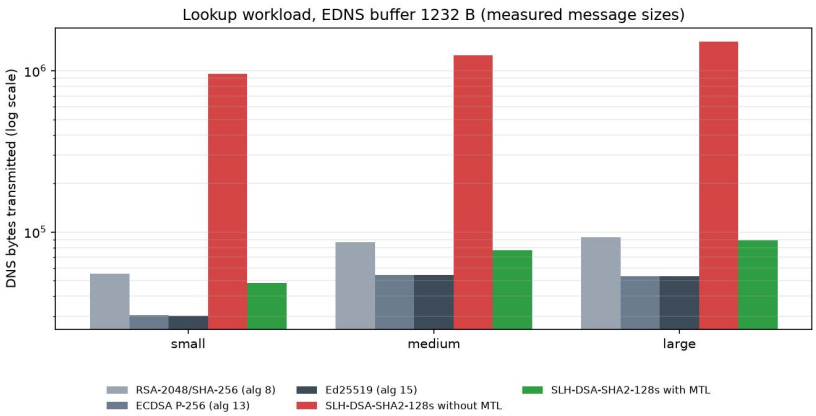
Quick demo run, log scale. Source: responses.csv, every answer measured once (median, min–max).

Say:

Response sizes by record class for the large zone, with the 1,232-byte UDP limit as a dotted line. Every direct SLH-DSA answer is around 8 kilobytes, and NXDOMAIN answers over 30 kilobytes because they carry up to four signatures: none fit. With MTL, answers for apex, small and medium records sit at a few hundred bytes, close to RSA, and all fit. What still exceeds the limit with MTL are the large TXT records, which exceed it with every algorithm, and some NXDOMAIN answers that carry several condensed signatures.

Bytes on the wire: 17× less than direct SLH-DSA

Total DNS bytes for the 301-lookup stream at a 1,232-byte buffer.



Large zone, 106 answers sent

- SLH-DSA direct **1,522,371 B**, 106 TCP retries
- SLH-DSA + MTL **88,895 B**, 33 TCP (32 truncated + 1 ladder fetch)
- Ed25519 53,292 B, 7 TCP · RSA 92,610 B, 34 TCP
- MTL ≈ 1.7× Ed25519 and below RSA-2048.

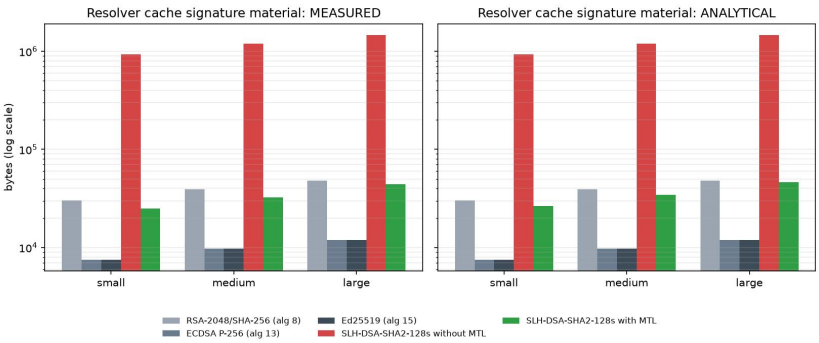
Quick demo run, log scale. Source: workload.csv, one deterministic 301-lookup replay per zone and mode.

Say:

Total bytes for the whole lookup stream. Direct SLH-DSA sends one and a half megabytes for 106 answers, and every single answer needs a TCP retry. MTL sends 89 kilobytes, 17 times less, including the one 8-kilobyte ladder fetch; 32 answers still need TCP, mostly the large TXT and some NXDOMAIN. That is within a factor of two of Ed25519 and slightly less than RSA-2048 today. In other words, MTL brings post-quantum DNSSEC traffic back to the range DNS already handles.

Cache and storage: 36× less than direct SLH-DSA

Signature bytes a resolver holds after the workload: measured (left), draft formulas (right).



Quick demo run, log scale. Source: cache.csv and sizes.csv.

Large zone

- Resolver cache: direct **1.47 MB** · MTL **44 KB** (incl. 8 KB ladder) · Ed25519 12 KB
- Zone signature storage: direct **3.18 MB** · MTL **0.087 MB** · Ed25519 0.026 MB
- Measured MTL sizes match the draft: condensed 204 B = formula 204 B; ladder 8,056 B ≤ 8,216 B max.

Say:

Cache usage. On the left, measured: the answers the resolver received are loaded into dnspython's real cache and we count the signature bytes it holds. Direct SLH-DSA needs one and a half megabytes for the large zone; MTL needs 44 kilobytes including the ladder, the same order as RSA. On the right, the same quantity from the formulas in section 12 of the MTL draft; measured and analytical agree, which also validates that the reference implementation produces the sizes the draft predicts. For the signer, storing all signatures for the zone drops from 3.18 megabytes to 87 kilobytes, 36 times less.

Slide 20 / 23

The result in one table

Large quick-demo zone, 405 record sets, medians of 3 trials.

	SLH-DSA direct	SLH-DSA + MTL	Ed25519 (today)
Signature per RRSIG	7,856 B	≤ 204 B + one 8,056 B ladder per zone	64 B
Sign whole zone	≈ 209 s	**0.52 s**	0.014 s
Verify one answer, ladder cached	588 µs	**5.3 µs**	111 µs
Verify first answer, nothing cached	511 µs	513 µs	103 µs
Small answers within 1,232 B	0%	**100%**	100%
Bytes on the wire, 301 lookups	1,522,371 B	**88,895 B** (17× less)	53,292 B
Signature storage for the zone	3.18 MB	**0.087 MB** (36× less)	0.026 MB
All correctness checks	passed	passed	passed

Conclusion: MTL makes SLH-DSA practical for DNSSEC except at first contact, where one ladder fetch and one SLH-DSA verification remain.

Say:

This table is the whole result. Against direct SLH-DSA, MTL signs the zone about 400 times faster, verifies a cached answer about 100 times faster, makes small answers fit in UDP again, cuts traffic 17 times and storage 36 times. Against today's Ed25519 it is close on traffic and faster on warm verification. The price is the first contact with a zone: one 8-kilobyte ladder fetch over TCP and one SLH-DSA verification. And every correctness check passed in every trial. These are quick-demo numbers; the full PS-036 run with zones up to 5,000 names and five trials replaces them.

What is measured, what is modelled

Every number is labelled so a reviewer knows how much to trust it.

Measured directly	Modelled or estimated
Signing and verification time of every library call (ns, pinned core)	Direct SLH-DSA zone totals on medium / large: sample of 60 / 100 × RRsets
Exact byte size of every DNS query and answer (dnspython encoding)	Transport: UDP / TC / TCP exchange sums; IP fragments and TCP segments by arithmetic
Signature and ladder sizes from the reference library	Resolver behaviour: cache holds everything (TTL 3600), one ladder per zone
Resolver cache contents (dnspython LRUCache, tracemalloc)	Draft §12 maxima shown next to measured

Known limitations

- DNSKEY holds a placeholder key of the right length (keys are made inside each trial), so the zone is size-exact but not validatable by an outside resolver. maxrss reports the Python parent, not mtlbench.
- Quick demo: 3 trials, ≤ 200 names. Full run: 5 trials, 50 / 500 / 5,000 names, plus optional NSD cross-check and packet capture.

Say:

It is important to be clear about what is measured. Timings are measured, call by call, on a pinned core. Message sizes are exact, because they come from real DNS encoding of real signatures. What is modelled is the transport arithmetic around those messages, and the resolver's behaviour: it caches everything for the run and fetches one ladder per zone, which is how the draft describes it. For direct SLH-DSA on the bigger zones, the zone total is extrapolated from a sample, and the charts mark that with hatching. Two limitations to state openly: the DNSKEY record contains a placeholder key of the correct length, so sizes are exact but an outside validator could not check the zone, and the maximum-memory figure reflects the Python parent process. The full PS-036 run adds bigger zones, more trials, and an optional cross-check against a real NSD server.

Slide 22 / 23

Reproduce, inspect, learn

Every number can be regenerated and traced to the line of code that produced it.

Want to…	Use
Run the benchmark	.\scripts\run_benchmark.ps1 [-Config configs/quick.yaml]
See every variable of stages 1–7 on tiny zones	own.py in the container → results\own_trace.txt
Look up any output file or column	docs\outputs-reference.md
Follow inputs → outputs stage by stage	docs\end-to-end-flow.md
Read the full report	results\runs\<run_id>\report.html
Explore interactively	streamlit run dashboard/app.py
Method and design	docs\methodology.md, docs\architecture.md

Say:

Everything shown today can be reproduced with one command, and every number can be traced. For learning, the own.py script runs the whole pipeline on tiny zones and prints every variable at every stage. The outputs reference explains every file and column with its unit and formula, and the end-to-end flow document lists the inputs and outputs of each stage with exact values. The report and the dashboard present the results of any run.

Thank you

Questions?

Akash Joshi · akashjoshi2023@gmail.com · A3-PS036-TC161

Say:

Thank you. Happy to take questions: on the design, on any stage, or on any number; each one can be traced to its file and the code that produced it.